hashlock.



2026

Penetration Testing

PYRON (DeFi)

June 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	34
Our Methodology	35
Disclaimers	37
About Hashlock	38

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The PYRON team partnered with Hashlock to conduct a security Penetration Test of their PYRON Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.
With the exception of one acknowledged Low Severity finding, all reported issues have been remediated prior to the publication of this report.

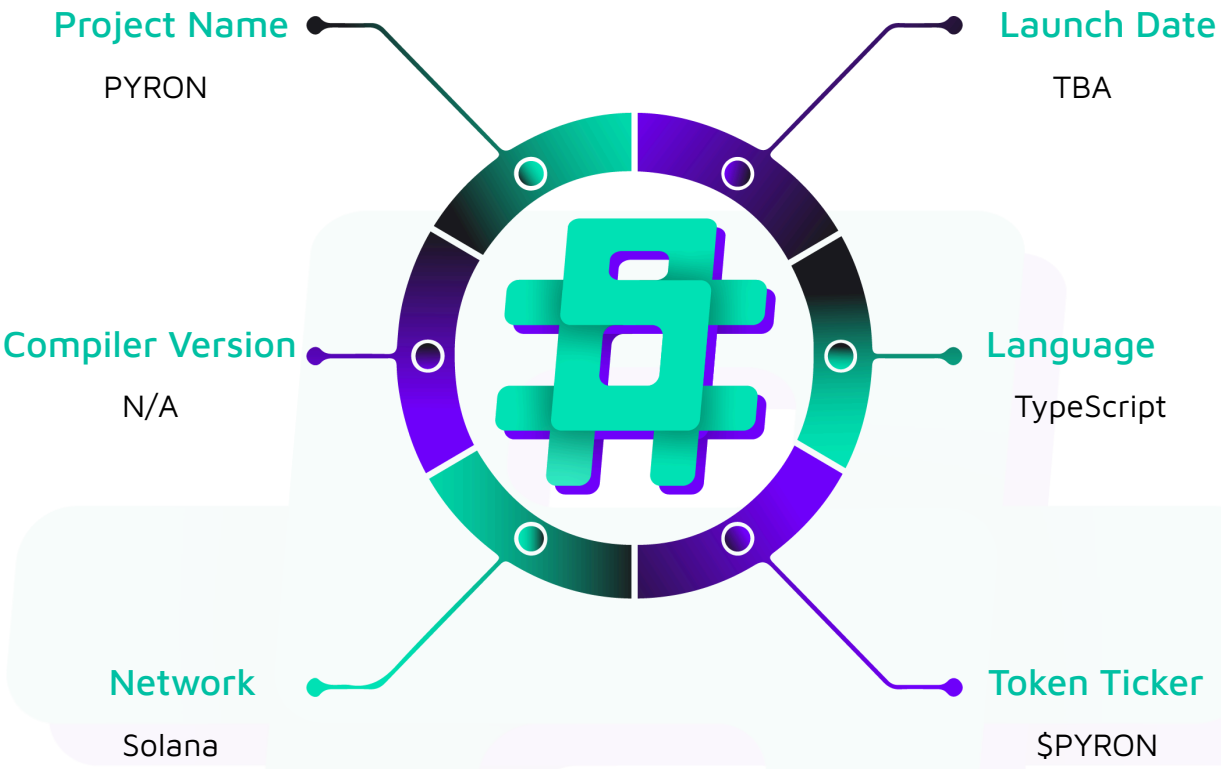
Project Context

PyRon employs a self-custodial architecture that ensures users retain exclusive control over their assets at all times. Unlike centralized exchanges, user funds are held in isolated on-chain accounts and are never commingled with the assets of other users. Every balance and transaction is transparently recorded on-chain, and only the user's authorized wallet can access or move those funds, meaning PyRon administrators cannot arbitrarily modify balances or access user assets. This architecture also provides platform independence: even if the PyRon platform becomes unavailable or the company ceases operations, users retain full access to their assets. In the event of a PyRon outage, users can directly manage their accounts through Squads. If Squads also becomes unavailable, the underlying infrastructure is open source, allowing anyone to deploy and interact with it independently, ensuring continued access to user funds.

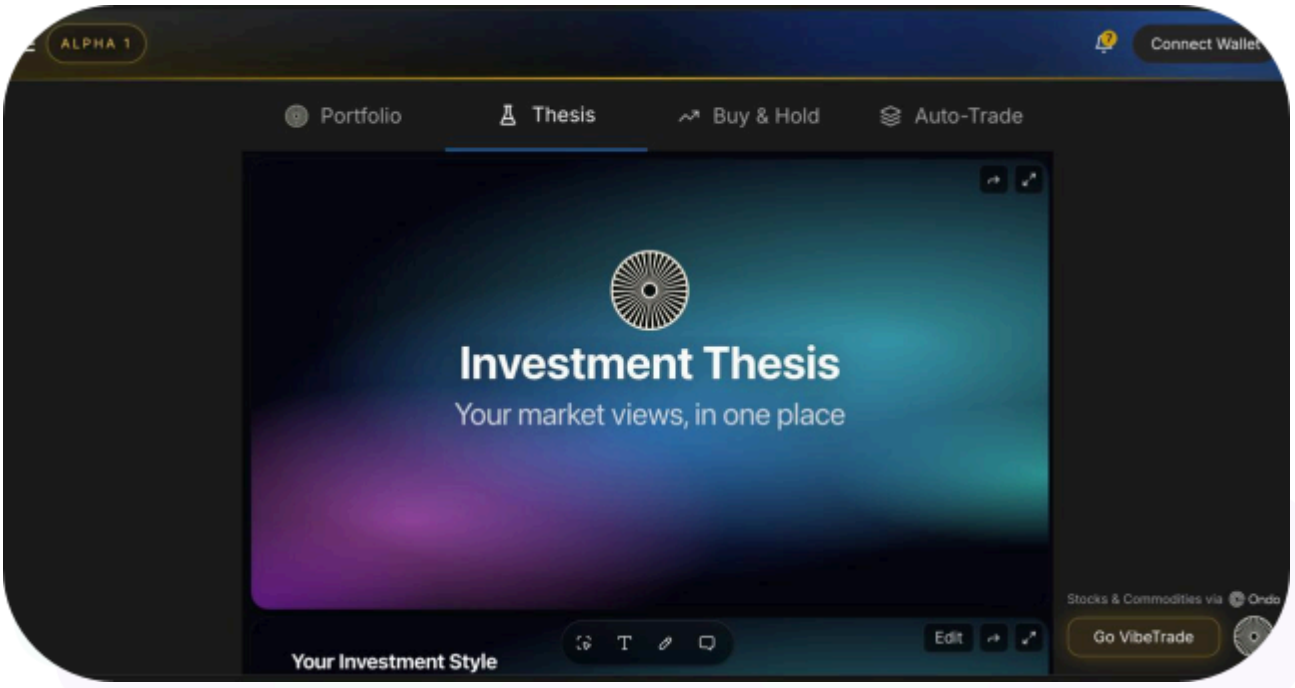
Project Name: PYRON
Project Type: Defi
Website: https://pyron-ai.com/
Logo:

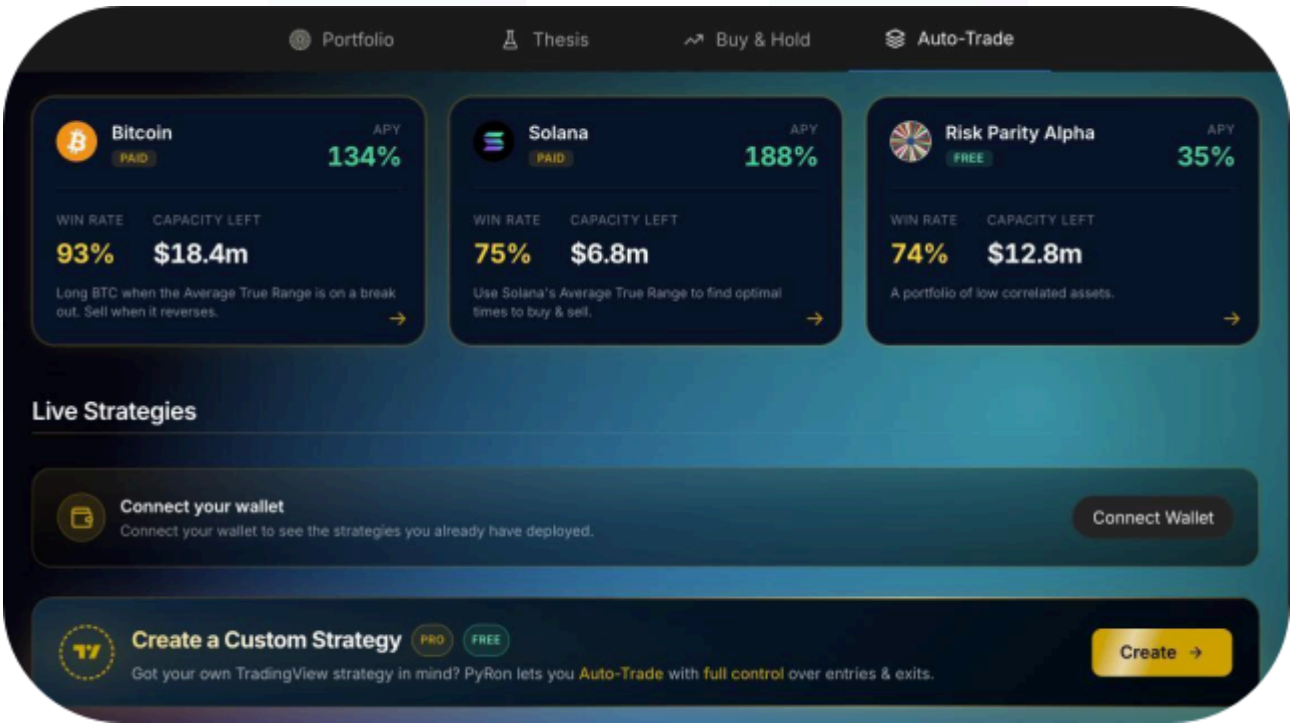


Visualised Context:



Project Visuals:





Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the PYRON project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**PYRON Project code base**
Platform	**TypeScript/Web apps & APIs**
Penetration Test Date	**June, 2026**
Repository/Website URL/IP Tested	https://github.com/UltraNova877/Meta-Repo-New-
GitHub Commit Hash (If applicable)	webApp: 43383b9, mvp: 832db09, trade-core: 3a2d1a7, webhook: d3a156d, webhook-mirror: 399f698
Fix Review GitHub Commit Hash (If applicable)	2637e225b3bed66bd0c32fc5478cc8640f596248

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High-severity vulnerabilities

3 Medium-severity vulnerabilities

8 Low-severity vulnerabilities

1 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01][pyron-mvp] controller/guardController.ts#buildDepositIx - an unverified `botId` lets one tenant overwrite another tenant's agent token-account records

Description

The `/deposit-ix` and `/withdraw-ix` handlers verify ownership of `multisigPda` and that `userWallet` equals the authenticated wallet, but never check that the separately supplied `botId` belongs to the caller. One tenant can therefore overwrite another tenant's agent records.

Vulnerability Details

The `buildDepositIx` handler derives `vaultPda` and guard accounts from the caller-owned `multisigPda`, mints fresh token-account keypairs owned by that vault, then persists them onto an agent selected only by the attacker-chosen `botId`. The lookup and update run as `Agent.findOne({ botId })` and `Agent.updateOne({ botId }, ...)` with no portfolio or owner filter. The `multisigOwnershipMiddleware` only proves the caller owns the multisig, never that the `botId` agent belongs to the same portfolio, unlike sibling agent routes which apply `agentOwnershipMiddleware`. The same horizontal-IDOR surface is observable client-side, where `pubkey`, `portfolioId`, `botId`, and `agentId` are interpolated into request paths with no principal binding.

```
const { multisigPda, userWallet, amountMicro, botId } = req.body;
const agent = await Agent.findOne({ botId }); // guardController.ts:391 - no
owner filter
await Agent.updateOne({ botId }, { $set: dbSet }); // guardController.ts:684 -
writes vault-derived accounts
```

Impact

An authenticated user who owns multisig A can submit a victim's `botId`, and the server overwrites the victim agent's `squadsAsset` token-account pubkeys with addresses derived from the attacker's vault. This corrupts the victim's reported PnL and position and de-links the agent from its real on-chain funds.

Recommendation

Add `agentOwnershipMiddleware` to the guard deposit and withdraw routes, or scope the agent lookup and update by the caller's portfolio (`Agent.findOne({ botId, portfolioId: ownedPortfolioId })`) so a non-owned `botId` cannot be mutated. The backend must also derive the principal from the session for every client-supplied identifier.

Status

Resolved

Medium

[M-01][pyron-mvp] controller/authController.ts#refreshToken - refresh tokens are never rotated and revocation is held only in a process-memory map

Description

The `/api/auth/refresh` endpoint issues a new access token but never rotates the refresh token and keeps no allowlist or family table, so a captured refresh token is replayable for its full lifetime. Revocation and logout state live only in a module-level in-process `Map`, so they are lost on restart and not shared across instances.

Vulnerability Details

The `refreshToken` handler reads `req.cookies.refresh_token`, verifies it, and returns only a fresh access token. The presented refresh token is not reissued and stays valid until its natural `exp` (default ~7 days). The only server-side state is the `revokedTokens` denylist `Map` at `authController.ts:10`, written on logout and checked on refresh, and `authMiddleware` never consults a denylist, so already-issued access tokens cannot be revoked. The `redisClient` exists but is not imported by the auth stores.

```
// controller/authController.ts:191-233 (refreshToken)
const payload = jwt.verify(refreshTokenValue, publicKey, { algorithms: ["RS256"]
});
// issues a NEW access token only; refresh_token is NOT rotated/reissued
const accessToken = signAccessToken(payload.walletAddress);
```

Impact

A refresh token captured through a log leak, device compromise, or backup exposure can be replayed to mint access tokens for up to its full TTL, and a logged-out token

becomes valid again after a process restart, with no rotation or reuse detection to cut the session short.

Recommendation

Rotate the refresh token on every `/refresh` call by issuing a new token and invalidating the old one, and maintain a per-family token-id store in Redis (a `jti` denylist with TTL keyed on token `exp`) so a replayed or old token triggers full-family revocation.

Status

Resolved

[M-02][pyron-trade-core / PyRon-webApp] src/jupiter/jupiterPlaceOrder.ts / src/services/platformService.ts - swap slippage controls are inadequate, leaving swaps exposed to sandwich and MEV extraction

Description

The trade-core SDK defines slippage bounds but does not enforce them before passing caller-controlled slippage to Jupiter, and the web app's liquidation-style swap paths use a hardcoded slippage value instead of threading the agent/user-configured tolerance into those flows.

Vulnerability Details

In trade-core, SLIPPAGE_TOLERANCE.MIN and .MAX are defined in src/constants.ts, but jupiterPlaceOrder.ts accepts any finite slippageTolerance and converts it directly to basis points. That value is passed into Jupiter quote construction and becomes part of the slippage-adjusted minimum output. The web app's swapAgentAssetToUSDC helper defaults to 300 bps, but the currently observed auto-trade disable and asset-change call sites pass a hardcoded 50 bps. This is not user-controlled and does not use the saved agent slippageTolerance.

```
onst slippagePct = (typeof slippageTolerance === "number" &&
isFinite(slippageTolerance)
    ? slippageTolerance
    : SLIPPAGE_TOLERANCE.DEFAULT);
  const slippageBps = Math.round(slippagePct * 100);
```

Impact

A compromised or buggy upstream caller of trade-core can widen slippage beyond the intended maximum, weakening price protection. In the web app, users cannot tune the

slippage used for forced asset-to-USDC flows, which may cause either unnecessary failed swaps in volatile markets or a mismatch between displayed/configured risk preferences and actual execution behavior.

Recommendation

Clamp slippageTolerance to SLIPPAGE_TOLERANCE.MIN and .MAX before converting to bps, and thread the agent/user-configured slippage tolerance into webapp liquidation and asset-change swap paths.

Status

Resolved

[M-03][pyron-mvp / PyRon-webApp] platforms/jupiterAdapter.ts / src/utils/tokenPrice.ts - third-party prices are consumed without staleness or sanity validation and feed PnL accounting

Description

Prices fetched from Pyth, Yahoo, Jupiter, and CoinGecko are accepted with no publish-time, staleness, confidence, or sanity-bound checks before being used in portfolio value and PnL calculations. The web app also contains a fallback transfer memo path that can write a USD value derived from a third-party price, although the currently observed liquidation call sites do not enable that fallback.

Vulnerability Details

In pyron-mvp, `getTokenPrice` and `getStockPrice` decode price and exponent and feed `positionValue = assetBalance * currentPrice` directly with at most a `typeof === 'number'` check (`jupiterAdapter.ts:120-298`). In PyRon-webApp, getUsdPriceForMint queries Coingecko over HTTPS with no validation, and jupiterSwap.ts:236-246 can multiply the returned price by the balance and write it into a WITHDRAWAL:AMOUNT memo when the fallback transfer path is enabled. The transferred token amount itself is the exact on-chain balance and is not derived from the price.

```
// src/utils/jupiterSwap.ts:236-246
const price = await getUsdPriceForMint(assetCfg.mint.toString(),
assetCfg.symbol);
if (typeof price === "number" && isFinite(price) && price > 0) {
  memoStr = SquadsMemoUtils.createWithdrawalMemo(uiBalance * price);
}
```

Impact

A manipulated or stale third-party price corrupts the displayed `positionValue` and `pnlUsd` and the recorded on-chain PnL memo value. It does not change the transferred amount, so the impact is accounting and display integrity rather than direct fund loss unless a downstream rule engine auto-trades on it.

Recommendation

Validate the Pyth `publish_time` and confidence and reject stale or low-confidence reads, add minimum and maximum sanity bounds with cross-source agreement before using a price, and treat the memo USD value as best-effort metadata.

Status

Resolved

Low

[L-01][pyron-mvp/pyron-trade-core] controller/guardController.ts#getExecutor / src/config.ts#initTradingCore - executor and admin signing secrets are held as plaintext in process memory with no KMS or HSM

Description

The backend co-signer keypair and the trade-core executor keypairs are reconstructed in process memory from plaintext base58 secrets with no KMS, HSM, or remote-signer abstraction and no rotation hook. The Privy `appSecret` that authorizes admin signing is held the same way.

Vulnerability Details

The `getExecutor()` function decodes `process.env.EXECUTOR_PRIVATE_KEY` into a full `Keypair` that co-signs guard transactions at `guardController.ts:162` and `guardController.ts:332`. In trade-core, `executor1` and `executor2` are typed as plain `string` in `src/config.ts:33-34`, decoded through `createKeypairFromSecretKey`, and used to sign every swap (`vtx.sign(executors)` at `onchainSwap.ts:499`). The Privy `appSecret` is likewise a plaintext field used for HTTP Basic auth in `adminSigner.ts:38`. Any env dump, heap dump, verbose crash log, or compromised dependency that can read process memory exposes the complete signing material.

```
// controller/guardController.ts:94-98
function getExecutor(): Keypair {
  const key = process.env.EXECUTOR_PRIVATE_KEY;
  if (!key) throw new Error("EXECUTOR_PRIVATE_KEY env var not set");
  return Keypair.fromSecretKey(bs58.decode(key));
}
```

Impact

Increased likelihood of leak of the most critical secrets.

Recommendation

Move the executor and admin signing keys behind a KMS, HSM, or remote signer that exposes a sign-only interface (for example AWS KMS, GCP KMS, or Turnkey), keep raw key material out of `process.env` and out of the frozen config object, and add a documented rotation procedure.

Status

Acknowledged

[L-02] [pyron-mvp / pyron-trade-core / PyRon-webApp] outbound fetch and Connection call sites - outbound HTTP and RPC calls lack timeouts and abort signals across all three services

Description

Outbound price-oracle, RPC, and backend-API calls are issued with no `AbortController` or timeout in all three components, so a slow or hung upstream holds the request open indefinitely. Several paths also page or fan out over externally controlled lists with no upper bound.

Vulnerability Details

In pyron-mvp, `jupiterAdapter` calls Pyth, Yahoo, Jupiter, and CoinGecko with bare `fetch` and no signal, and `guardController.getConnection` builds the `Connection` with no timeout while the PnL scanners page `getSignaturesForAddress({ limit: 1000 })` in unbounded loops. In trade-core, `getPriorityFee` (`getPriorityFees.ts:13-30`) has no abort signal and `onchainSwap.ts:297-303` issues one serial RPC per Jupiter-supplied ALT address with no cap. In PyRon-webApp, `apiFetch` (`apiClient.ts:43-48`) and the raw guard, auth, and custody fetches omit `signal`, unlike the sibling Jupiter calls that set `AbortSignal.timeout(10000)`.

```
// platforms/jupiterAdapter.ts (representative)
const resp = await
fetch(`https://hermes.pyth.network/api/latest_price_feeds?ids[]=${id}`, {
  headers: { Accept: "application/json" },
}); // no AbortController / no timeout
// src/services/apiClient.ts:43-48
const response = await fetch(`${API_BASE_URL}${endpoint}`, { headers,
credentials: "include" });
```

Impact

A degraded upstream can pile up in-flight requests against worker and socket resources, degrading availability, and a hung backend leaves user-blocked deposit, withdraw, and auth flows pending with no client-side cap, retry bound, or circuit breaker.

Recommendation

Wrap every outbound `fetch` and `Connection` in an `AbortController` with a per-call timeout (mirror the existing 10s pattern), add a default `AbortSignal.timeout(...)` in `apiFetch`, cap the number of history pages and ALT addresses processed per request, and add a short price cache plus a per-upstream circuit breaker.

Status

Resolved

[L-03][pyron-webapp] src/store/walletStore.ts / src/utils/tokenPrice.ts - the wallet RPC and the price provider have no fallback or failover

Description

The wallet store builds a single `Connection` from `VITE_RPC_URL` with no fallback, and the price helper queries only Coingecko despite a comment claiming multiple providers, so a single provider outage breaks on-chain reads or the fallback memo flow.

Vulnerability Details

The `walletStore.ts:54` line uses `new Connection(import.meta.env.VITE_RPC_URL, "confirmed")` with no `||` fallback and no per-call timeout, whereas `WalletAdapterProvider.tsx:44` does fall back to public mainnet, an inconsistency that leaves balance reads, blockhash fetches, confirmations, and submission dependent on one endpoint. The `getUsdPriceForMint` helper has no Jupiter or Birdeye fallback and returns null on any failure.

```
// src/store/walletStore.ts:54
const connection = new Connection(import.meta.env.VITE_RPC_URL, "confirmed");
```

Impact

A single RPC outage or rate-limit breaks balance reads, simulations, confirmations, and submissions across deposit, withdraw, and funding flows with no automatic recovery, and a single Coingecko outage blocks the disable and asset-change fallback memo entirely.

Recommendation

Centralize a connection factory with a validated endpoint plus a fallback list and bounded retry and failover, align both call sites, and implement the documented multi-provider price fallback.

Status

Resolved

[L-04][pyron-webapp] index.html / nginx deploy config - the deployment sets no Content-Security-Policy and no HSTS header

Description

Neither `index.html` nor the nginx deployment config defines a Content-Security-Policy, and the TLS-terminating nginx sets no Strict-Transport-Security header.

Vulnerability Details

The nginx server block in the deploy workflows adds `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection`, and `Referrer-Policy` but no `Content-Security-Policy` and no `Strict-Transport-Security`, and `index.html` has no CSP meta tag.

```
# .github/workflows/deploy-prod.yml (nginx block)
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
# no Content-Security-Policy, no Strict-Transport-Security
```

Impact

The absent CSP removes the strongest defense against an XSS-to-wallet-drain in a wallet-bearing dApp, and the absent HSTS leaves the first request open to an SSL-strip or downgrade. No in-scope XSS sink was found, so the CSP gap is defense-in-depth.

Recommendation

Add a strict `Content-Security-Policy` (`script-src 'self'` with nonces or hashes) and `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` at the nginx layer.

Status

Resolved

[L-05][pyron-mvp] main.ts#authLimiter - auth endpoints are rate-limited per-IP only with no per-account lockout

Description

The `/api/auth/*` routes are protected by a single per-IP limiter with no per-account limit, lockout, or backoff on the nonce and token endpoints.

Vulnerability Details

The `authLimiter` at `main.ts:66` keys solely on IP, so a distributed set of IPs can grind nonce and token attempts against one target wallet without tripping the per-IP cap, and there is no exponential backoff or lockout after repeated failures. Because auth is Ed25519 wallet-signature, the practical risk is nonce-flood and resource amplification rather than credential brute-force.

Impact

A distributed attacker can flood nonce and token issuance for a targeted wallet or amplify resource use, though signature unforgeability prevents credential compromise.

Recommendation

Add a per-`walletAddress` limiter and a short lockout or backoff after repeated failed `/token` attempts, in addition to the per-IP cap.

Status

Resolved

[L-06][pyron-mvp] controller/guardController.ts#computeBudget - a static compute-unit limit may underprovision multi-account transactions

Description

A hardcoded 400k compute-unit limit and a fixed priority fee are applied to every guard transaction regardless of how many token-account create and initialize instructions are appended.

Vulnerability Details

The `computeBudget()` function at `guardController.ts:106-111` returns fixed `setComputeUnitPrice(5_000)` and `setComputeUnitLimit(400_000)` applied at several build sites, while deposit construction can append up to four create-and-initialize pairs for Token-2022 mints whose compute cost is state-dependent.

Impact

If the appended instructions push compute past 400k the transaction reverts out of compute units and the user pays fees for a failed deposit, a determinism and UX issue rather than a fund-loss vector.

Recommendation

Estimate compute units from the actual instruction set or simulate the transaction, and set the limit dynamically with a safety margin above the worst-case Token-2022 init path.

Status

Resolved

[L-07][pyron-trade-core] src/jupiter/onchainSwap.ts#guardSwapViaJupiter - an upstream API response body is returned in an unscrubbed error string

Description

On an axios failure the full upstream response body and HTTP status are stringified into the `error` field of the returned result object, which, unlike the logger path, is not run through `sanitizeSecrets`.

Vulnerability Details

The `onchainSwap.ts:339-343` block appends `Status: <status> | Data: ${JSON.stringify(axiosErr.response.data)}` to the error message and returns it. The logger path applies `sanitizeSecrets`, but the returned object does not, so the host decides where the raw string goes.

Impact

If a host logs the returned `error` verbatim outside this library's logger, upstream response bodies surface. In practice the failing calls hit the public keyless Jupiter quote endpoint whose error bodies are not sensitive, so disclosure impact is low.

Recommendation

Run `sanitizeSecrets` over the error message before returning it, not only when logging.

Status

Resolved

[L-08][PyRon-webApp] src/services/authService.ts / src/utils/agentUtils.ts - No client-side rate limiting on auth and signature endpoints

Description

The client imposes no throttle or backoff on the nonce, token, refresh, or generate-webhook-signature endpoints.

Vulnerability Details

While the 401 handler dedupes and bounds refresh to a single retry, there is no rate limiting on the auth and signature endpoints and `keepAliveTick` fires periodically. Rate limiting properly belongs at the backend and nginx layer, but the client provides no second-order defense.

Impact

The client provides no defense against brute-force or resource-exhaustion on backend auth and signature endpoints, a second-order denial-of-service consideration.

Recommendation

Add client-side debounce and backoff on auth and signature requests and confirm the backend rate limits.

Status

Resolved

QA

[Q-01][pyron-mvp] main.ts#errorHandling - Stack traces are logged and rate-limit IP keying depends on an absent X-Forwarded-For

Description

Server error handlers log full stack traces, and `express-rate-limit` keys on `req.ip` under `trust proxy 1`, but the production nginx config does not set `X-Forwarded-For`, so the per-IP limiter can collapse all clients into one bucket or become spoofable.

Vulnerability Details

Several controllers log `error.stack` on a 500. The `main.ts:47` line sets `app.set('trust proxy', 1)`, so `req.ip` is taken from the right-most `X-Forwarded-For` hop, but the inlined nginx block sets `CF-Connecting-IP` and `X-Forwarded-Proto` and not `X-Forwarded-For`. When the header is absent Express falls back to the socket IP, so all clients may share one bucket, and if a client supplies its own header the per-IP buckets become spoofable.

Impact

Client responses are sanitized, but stack traces in logs aid an attacker with log access, and the rate-limiter can be globally collapsed or per-IP-spoofed, weakening abuse protection.

Recommendation

Have nginx set `X-Forwarded-For $proxy_add_x_forwarded_for` or configure a `keyGenerator` using `CF-Connecting-IP`, and gate stack-trace logging behind `NODE_ENV !== 'production'`.

Status

Resolved

Conclusion

After Hashlock's analysis, the PYRON project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

